Program 1

Problem Statement
Use single inheritance from a base 'shape' class: a 'volume' derived class and an 'Area' derived class each compute volume / surface area for a Cube, Cylinder and Box from shared dimension data.

Source Code:

```
#include <iostream>
#include <string>
using namespace std;

// Base Class
class shape {
protected:
double side1, side2, side3;

public:
shape(double x, double y, double z) {
side1 = x;
side2 = y;
side3 = z;
}

void display() {
cout << "side1 is: " << side1 << endl;
cout << "side2 is: " << side2 << endl;
cout << "side3 is: " << side3 << endl;
}
};

// Derived Class for Volume
class volume : public shape {
protected:
string shapeType;
double Volume;

public:
volume(string w, double x, double y, double z) : shape(x, y, z) {
shapeType = w;
}

void CubeVolume() {
Volume = side1 * side1 * side1;
}

void cylinderVolume() {
Volume = 3.1416 * side1 * side1 * side2;
}

void boxVolume() {
```

```
Volume = side1 * side2 * side3;
}

void displayVolume() {
cout << "The Shape is: " << shapeType << endl;
cout << "Volume = " << Volume << endl;
}
};

// Derived Class for Area
class Area : public shape {
protected:
double area;
string shapeType;

public:
Area(string w, double x, double y, double z) : shape(x, y, z) {
shapeType = w;
}

void CubeArea() {
area = 6 * side1 * side1;
}

void cylinderArea() {
area = 2 * 3.1416 * side1 * (side1 + side2);
}

void boxArea() {
area = 2 * ((side1 * side2) + (side2 * side3) + (side3 * side1));
}

void displayArea() {
cout << "Area = " << area << endl;
}
};

int main() {
// 1. Cube Calculations
volume volume1("Cube", 12, 0, 0);
volume1.display();
volume1.CubeVolume();
volume1.displayVolume();

Area area1("Cube", 12, 0, 0);
area1.CubeArea();
area1.displayArea();

cout << endl; // Separator for clarity

// 2. Cylinder Calculations
```

```
volume volume2("Cylinder", 12, 25, 0);
volume2.display();
volume2.cylinderVolume();
volume2.displayVolume();

Area area2("Cylinder", 12, 25, 0);
area2.cylinderArea();
area2.displayArea();

cout << endl;

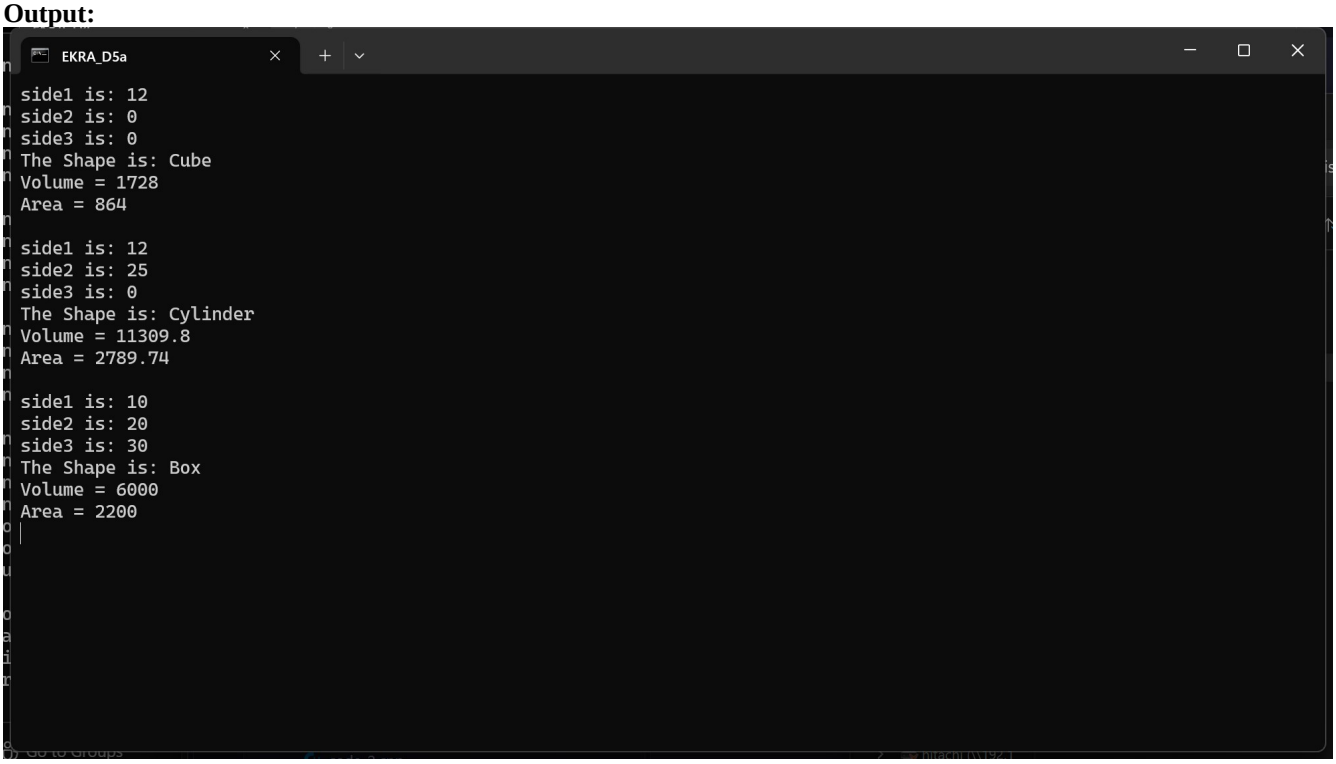
// 3. Box Calculations
volume volume3("Box", 10, 20, 30);
volume3.display();
volume3.boxVolume();
volume3.displayVolume();

Area area3("Box", 10, 20, 30);
area3.boxArea();
area3.displayArea();

return 0;
}
```

Input:
(no input)

Output:



Program 2

Problem Statement
Use multi-level inheritance (root -> calculation -> decision -> root_calculation) to compute the discriminant of a quadratic equation, state whether the roots are real or complex, and print the two roots for a = 2, b = 5, c = 3.

Source Code:

```
#include <bits/stdc++.h>
using namespace std;

class root {
protected:
int a, b, c;

public:
root(int num1, int num2, int num3) {
a = num1;
b = num2;
c = num3;
}

void display() {
cout << "Value of a: " << a << endl;
cout << "Value of b: " << b << endl;
cout << "Value of c: " << c << endl;
}
};

class calculation : public root {
public:
float D;

calculation(int a, int b, int c) : root(a, b, c) {
}

void D_calculation() {
D = pow(b, 2) - (4 * a * c);
}
};

class decision : public calculation {
public:
decision(int a, int b, int c) : calculation(a, b, c) {
}

void root_decision() {
if (D >= 0) {
cout << "real Number" << endl;
} else {
cout << "Complex number" << endl;
}
}
```

```
};

class root_calculation : public decision {
public:
root_calculation(int a, int b, int c) : decision(a, b, c) {
}

void calculation_root() {
// Note: The formula in the image is (-b +/- sqrt(D)) / 2 * a
// Due to operator precedence, this calculates as ((-b +/- sqrt(D)) / 2) * a
int x1 = (-b + sqrt(D)) / 2 * a;
int x2 = (-b - sqrt(D)) / 2 * a;

cout << "x1 = " << x1 << endl;
cout << "x2 = " << x2 << endl;
}
};

int main() {
root_calculation obj(2, 5, 3);
obj.display();
obj.D_calculation();
obj.root_decision();
obj.calculation_root();
return 0;
}
```

Input:
(no input)

Output:

